

CHAPTER 12.

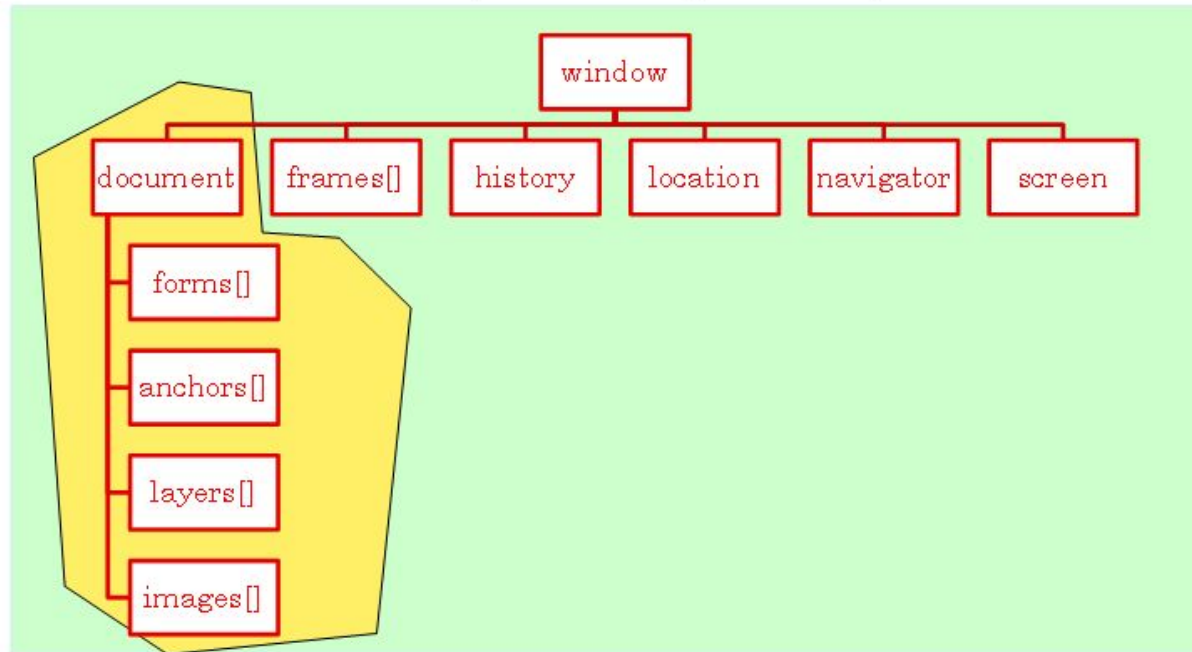
BOM, 이벤트 처리, 입력 검증



JS에서 사용할수있는 내장 객체종류

- HTML 문서를 객체로 표현한 것을 DOM
- 웹 브라우저를 객체로 표현한 것을 BOM(Browser Object Model)

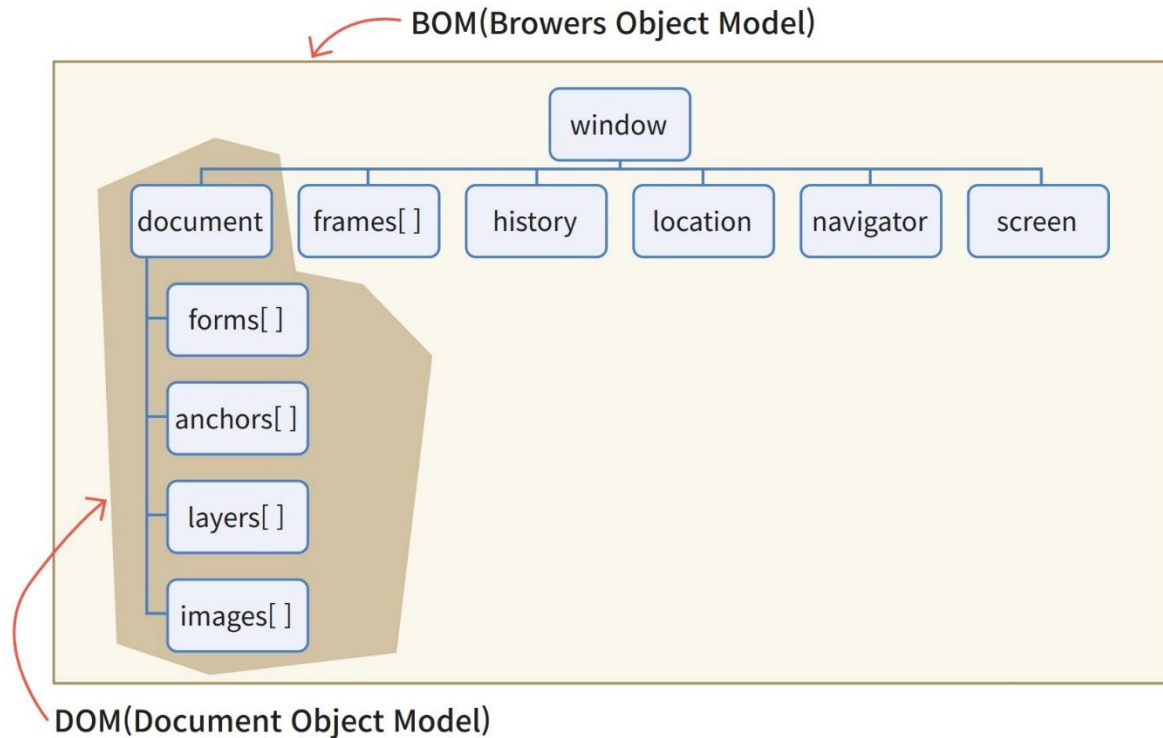
BOM(Browser Object Model)



DOM(Document Object Model)

브라우저 객체 모델

- 브라우저 객체 모델(BOM: Browser Object Model): 웹 브라우저가 가지고 있는 모든 객체를 의미
- 최상위 객체는 window이고 그 아래로 navigator, location, history, screen, document, frames 객체가 있다.

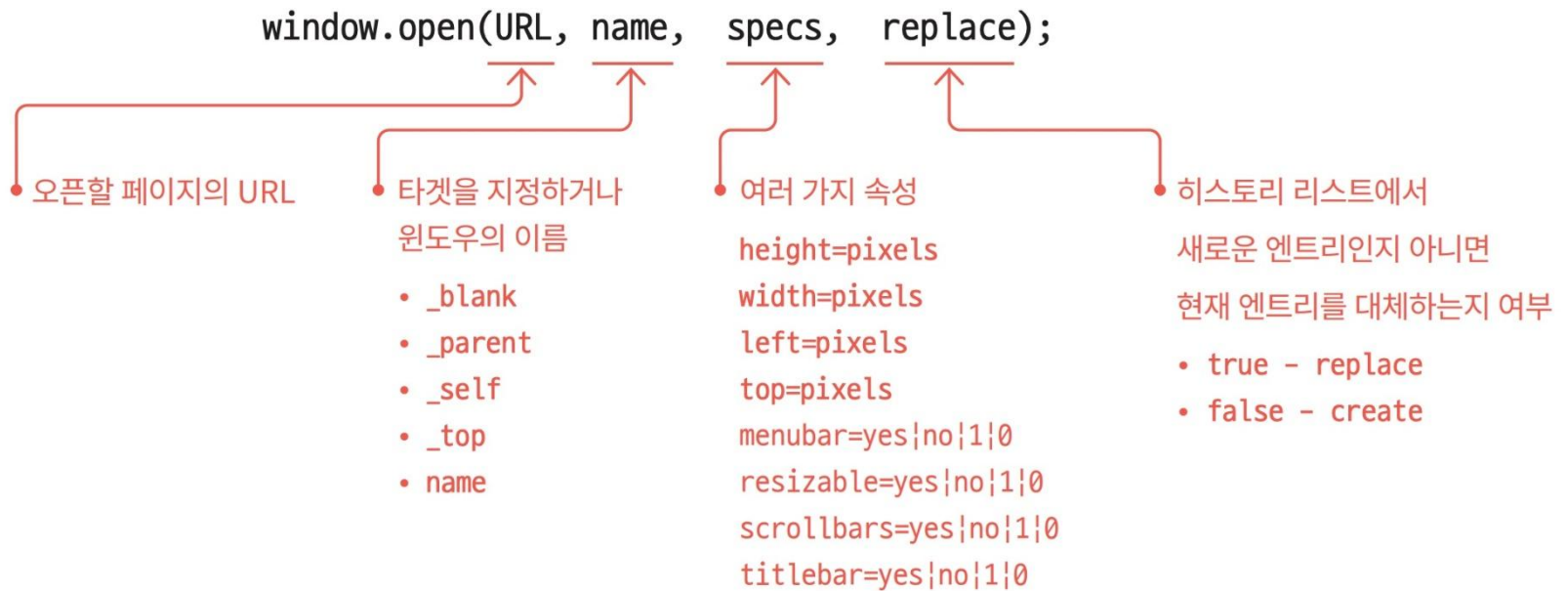


BOM에 포함된 객체

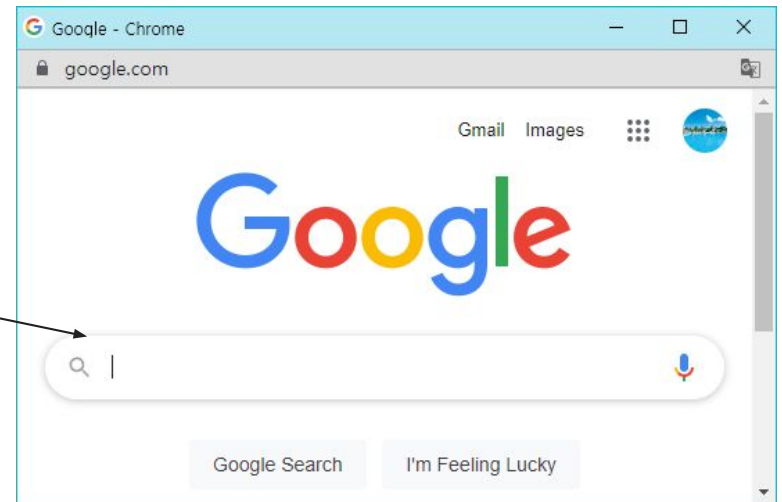
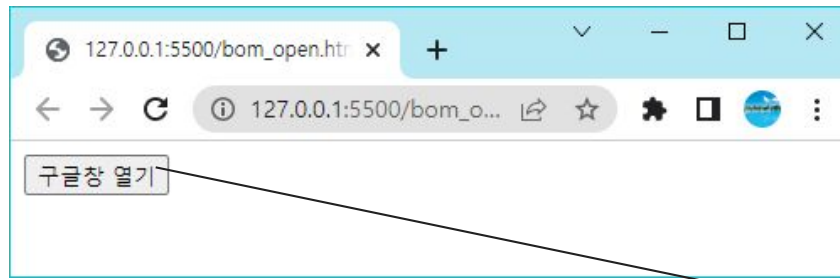
객체	설명
document	HTML 문서 정보
navigator	브라우저 자체에 대한 정보
screen	사용자 화면에 대한 정보
history	방문 기록 정보
location	현재 URL 정보
frames	윈도우에 포함된 프레임 정보

window 객체

- **window** 객체는 **BOM**에서 최상위 객체로서 웹브라우저 윈도우를 나타내고 있다. 자바스크립트의 모든 전역 함수는 **window** 객체의 메소드이다.



새로운 윈도우 오픈 예제

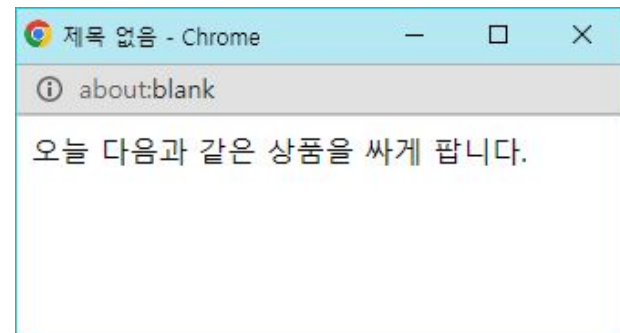
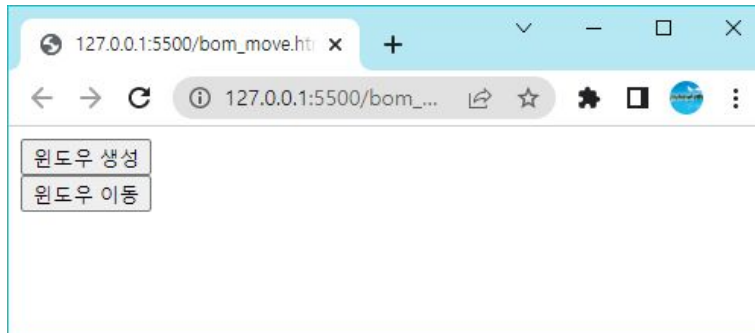


예제

```
<!DOCTYPE html>
<html>
<head></head>
<body>
  <form>
    <input type="button" value="구글창 열기"
      onclick="window.open('https://www.google.com', '_blank', 'width=300, height=300', true)">
  </form>
</body>
</html>
```

moveTo(), moveBy()

- 원도우를 이동하는 메소드이다. 절대적인 위치로 이동하는 것은 `moveTo()`를 사용하고 상대적으로 이동하려면 `moveBy()`를 사용한다.

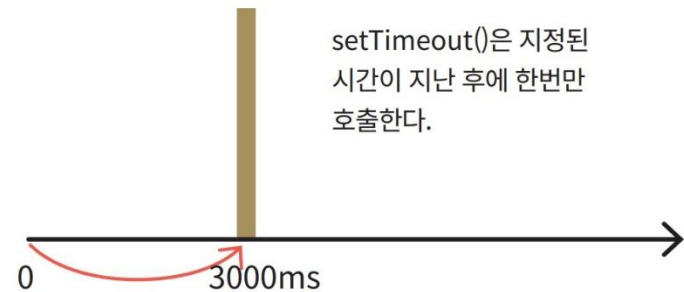
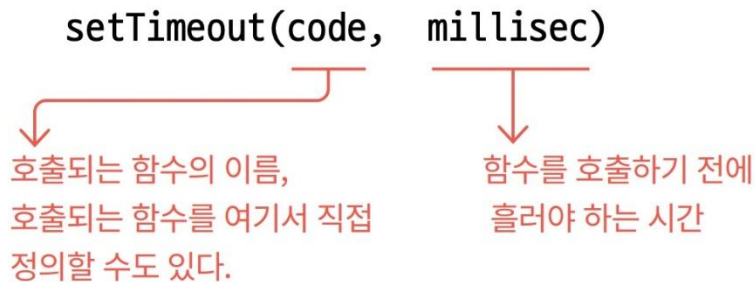


예제

```
<!DOCTYPE html>
<html>
<head>
<script>
  let w;
  function openWindow() {
    w = window.open(' ', ' ', 'width=200, height=100');
    w.document.write("<p>오늘 다음과 같은 상품을 싸게 팝니다.</p>");
    w.moveTo(0, 0);
  }
  function moveWindow() {
    w.moveBy(10, 10);
    w.focus();
  }
</script>
</head>
<body>
<input type="button" value="윈도우 생성" onclick="openWindow()" /> <br>
<input type="button" value="윈도우 이동" onclick="moveWindow()" />
</body>
</html>
```

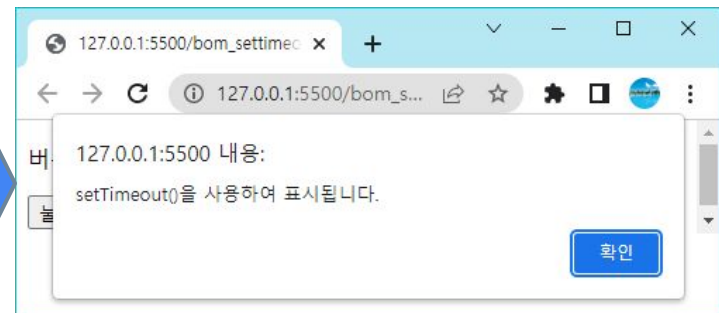
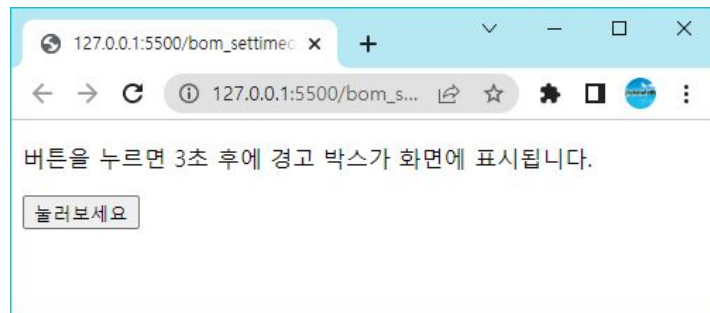
setTimeout()

- `setTimeout()` 메소드는 일정한 시간이 지난 후에, 인수로 전달된 함수를 한번만 호출한다. 시간의 단위는 밀리초이다.



예제:

- 버튼을 누르면 3초 후에 경고 윈도우를 오픈하는 예제를 작성하여 보자.

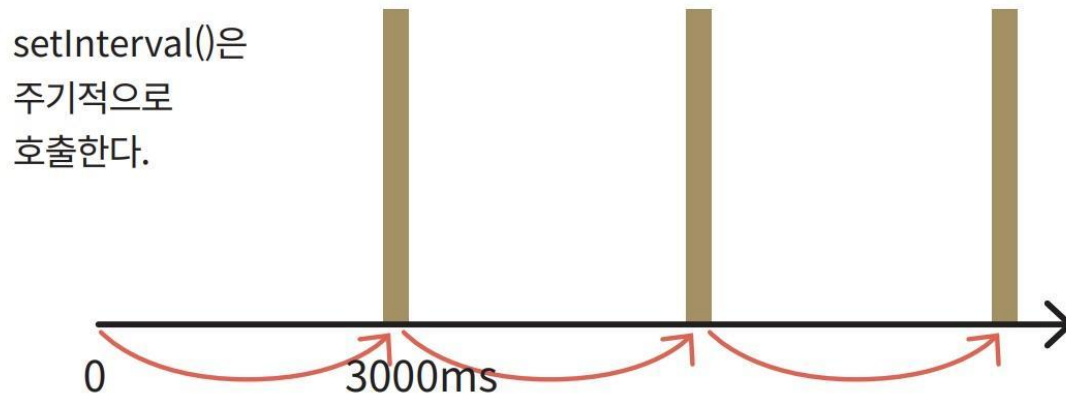


예제

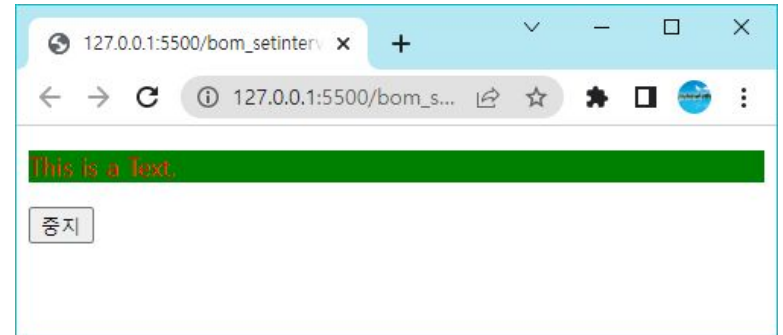
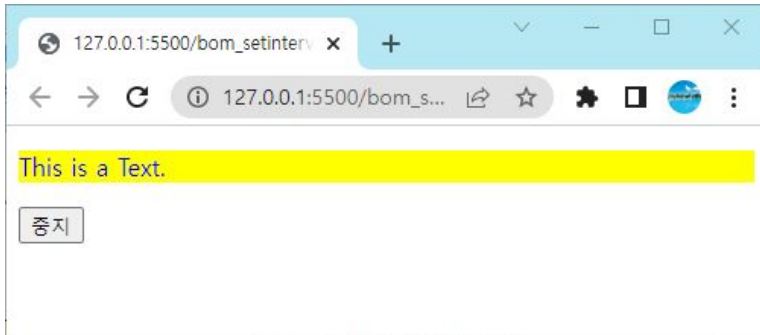
```
<!DOCTYPE html>
<html>
<head>
  <script>
    function showAlert() {
      setTimeout(function () { alert("setTimeout()을 사용하여 표시됩니다.") }, 3000);
    }
  </script>
</head>
<body>
  <p>버튼을 누르면 3초 후에 경고 박스가 화면에 표시됩니다. </p>
  <button onclick="showAlert()">눌러보세요</button>
</body>
</html>
```

setInterval()

- `setInterval()` 메소드는 일정한 시간마다 주기적으로 함수를 호출한다. `setInterval()`은 반드시 개발자가 종료시켜야 한다. 시간의 단위는 밀리초이다.



예제



```
<!DOCTYPE html>
<html>
<head>
  <script>
    let id;
    function changeColor() {
      id = setInterval(flashText, 500);
    }
    function flashText() {
      let elem = document.getElementById("target");
      elem.style.color = (elem.style.color == "red") ? "blue" : "red";
      elem.style.backgroundColor =
        (elem.style.backgroundColor == "green") ? "yellow" : "green";
    }
  </script>
</head>
<body>
  This is a Text.
  <button>종지</button>
</body>
</html>
```

예 제

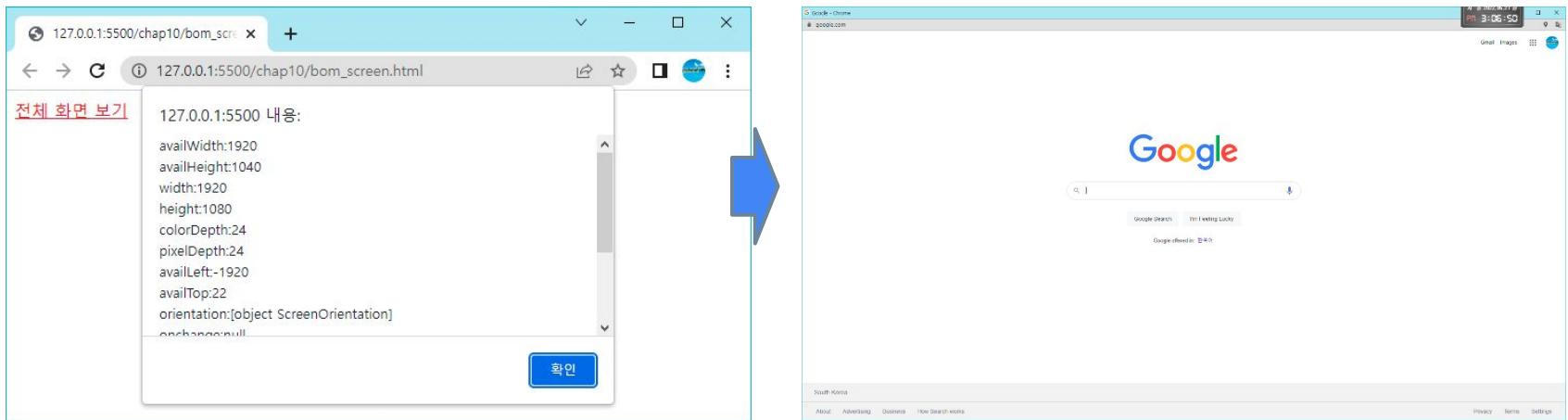
```
function stopTextColor() {  
    clearInterval(id);  
}  
</script>  
</head>  
  
<body onload="changeColor();">  
    <div id="target">  
        <p>This is a Text.</p>  
    </div>  
    <button onclick="stopTextColor();">중지</button>  
</body>  
</html>
```

screen 객체

속성	설명
availHeight	화면의 높이를 반환(윈도우에서 태스크바를 제외한 영역)
availWidth	화면의 너비를 반환(윈도우에서 태스크바를 제외한 영역)
colorDepth	컬러 팔레트의 비트 깊이를 반환
height	화면의 전체 높이를 반환
pixelDepth	화면의 컬러 해상도(bits per pixel)를 반환
width	화면의 전체 너비를 반환

예제

- **screen** 객체의 모든 속성을 출력해보고, 구글의 홈페이지를 로드하는 코드를 작성하면 다음과 같다.



예제

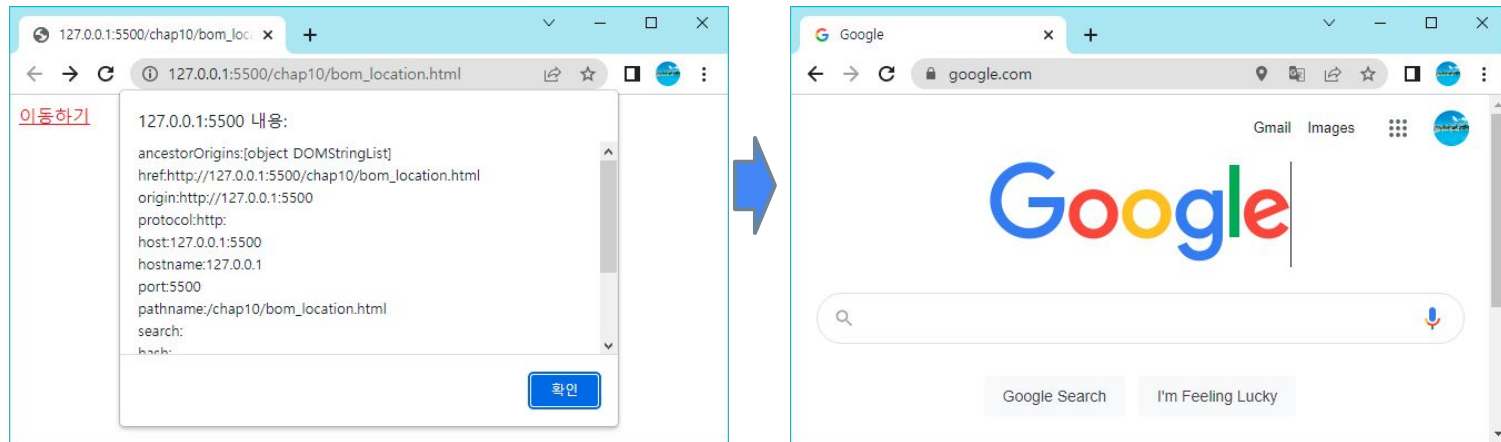
```
...
<script>
  function maxopen(url, winattributes) {
    let s = "";
    for (let key in screen) {
      value = screen[key];
      s += key + ":" + value + "\n";
    }
    alert(s)
    let maxwindow = window.open(url, "", winattributes)
    maxwindow.moveTo(0, 0);
    maxwindow.resizeTo(screen.availWidth, screen.availHeight)
  }
</script>
<a href="#"onClick="maxopen('https://www.google.com', 'resize=1, scrollbars=1, status=1'); return
false">전체 화면 보기</a>
```

location 객체

속성	설명
hash	URL 중에서 앵커 부분을 반환한다(#section1과 같은 부분).
host	URL 중에서 hostname와 port를 반환
hostname	URL 중에서 hostname을 반환
href	전체 URL을 반환
pathname	URL 중에서 경로(path)를 반환
port	URL 중에서 port를 반환
protocol	URL 중에서 protocol 부분을 반환
search	URL 중에서 쿼리(query) 부분을 반환
메소드	설명
assign()	새로운 문서를 로드한다.
reload()	현재 문서를 다시 로드한다.
replace()	현재 문서를 새로운 문서로 대체한다.

예제

- **location** 객체의 모든 속성을 출력해보고, **replace()**를 사용하여 구글의 웹페이지를 로드하는 코드를 작성해보자.



예제

```
...  
<script>  
  function replace() {  
    let s = "";  
    for (let key in location) {  
      value = location[key];  
      s += key + ":" + value + "\n";  
    }  
    alert(s)  
    location.replace("https://www.google.com")  
  }  
</script>  
<a href="#" onClick="replace()">이동하기 </a>
```

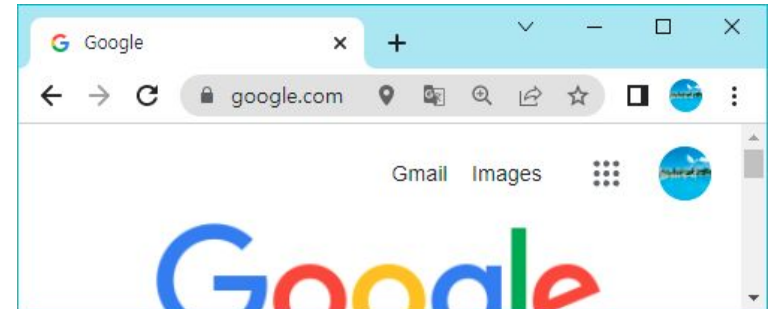
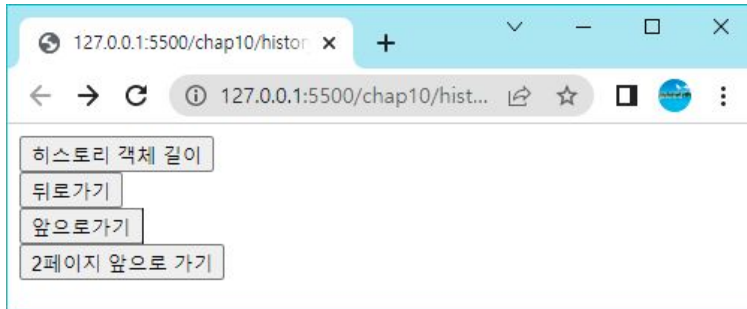
history 객체

- `window.history` 객체에는 브라우저 기록이 포함되어 있다. 몇 개의 메소드를 지원한다.

메소드	설명
<code>back()</code>	브라우저에서 “뒤로가기”를 클릭하는 것과 동일
<code>forward()</code>	브라우저에서 “앞으로 가기”를 클릭하는 것과 동일
<code>go(n)</code>	n번째 페이지로 가기

예제

- 버튼을 클릭하면 뒤로 가거나 앞으로 가는 예제를 작성해보자.



예제

```
<!DOCTYPE html>
<html>
<body>
  <input type="button" value="히스토리 객체 길이"
    onclick="alert(history.length);"/><br>
  <input type="button" value="뒤로가기" onclick="history.back()"/><br>
  <input type="button" value="앞으로가기" onclick="history.forward()"/><br>
  <input type="button" value="2페이지 앞으로 가기" onclick="history.go(+2)"/><br>
</body>
</html>
```

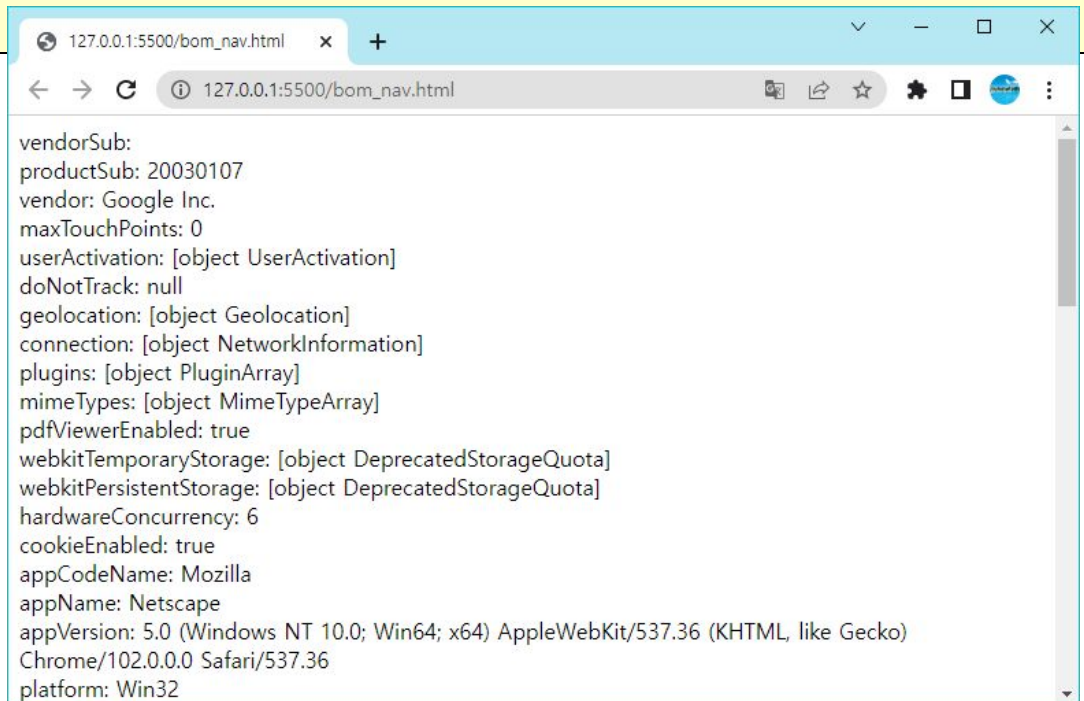

navigator 객체

속성	설명
appName	브라우저의 코드 네임
appName	브라우저의 이름
appVersion	브라우저의 버전 정보
cookieEnabled	브라우저에서 쿠키가 활성화되어 있는지 여부
onLine	브라우저가 인터넷에 연결되어 있으면 true
platform	브라우저가 컴파일된 플랫폼
userAgent	브라우저에서 서버로 가는 user-agent 헤더

메소드	설명
assign()	새로운 문서를 로드한다.
reload()	현재 문서를 다시 로드한다.
replace()	현재 문서를 새로운 문서로 대체한다.

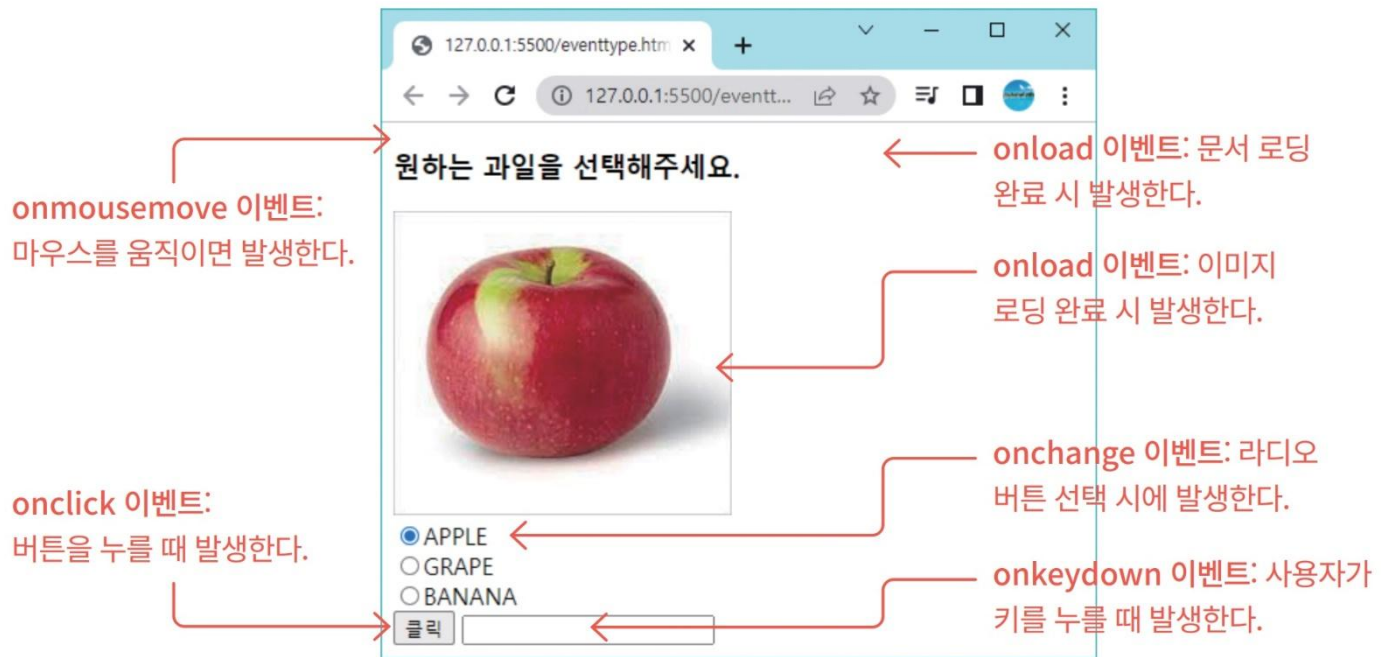
예제

```
<script>
  for (var key in navigator) {
    value = navigator[key];
    document.write(key + ": " + value + "<br>");
  }
</script>
```



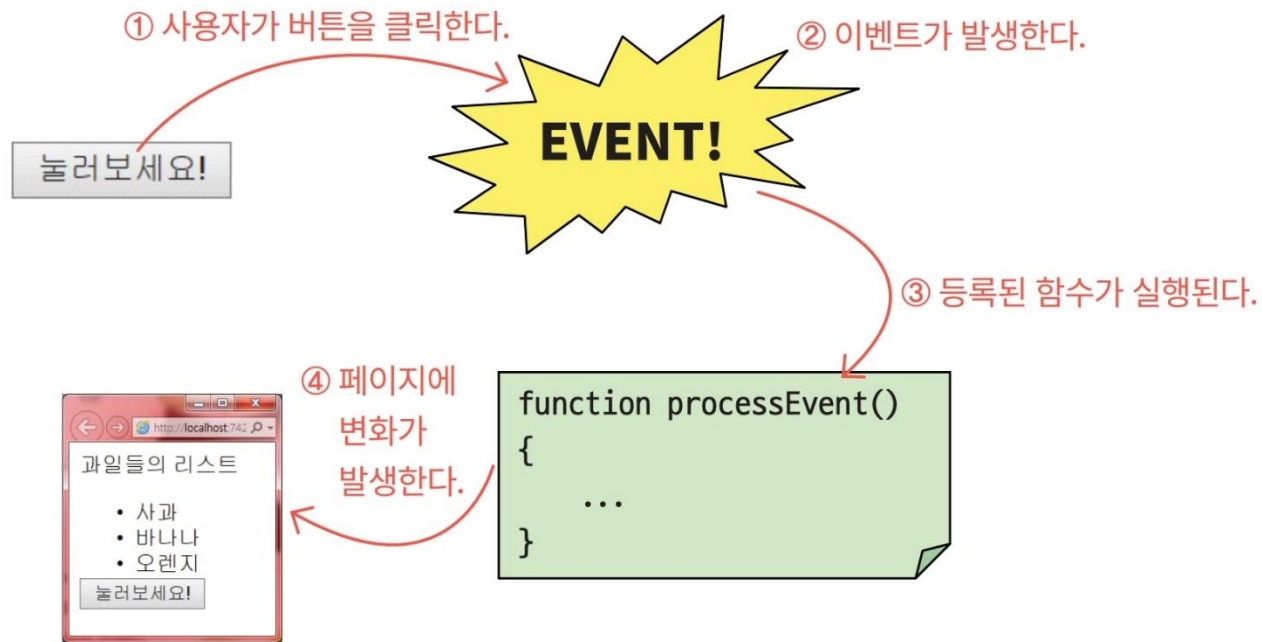
이벤트란 무엇인가?

- 웹 페이지에서 상호작용이 발생하면 이벤트가 일어난다. 예를 들어 사용자가 버튼을 클릭하거나 특정 요소 위로 마우스 커서를 가져가면 이벤트가 발생한다



이벤트 처리 절차

- 이벤트를 처리하는 코드를 버튼에 등록하여야 한다. 이벤트를 처리하는 코드를 이벤트 핸들러, 또는 이벤트 리스너라고 한다.



이벤트의 종류

- 마우스 관련 이벤트

이벤트	이벤트 리스너	설명
click	onclick	마우스로 요소를 클릭했을 때 발생한다.
mouseover	onmouseover	마우스 커서가 요소 위로 올 때 발생한다.
mouseout	onmouseout	마우스 커서가 요소를 떠날 때 발생한다.
mousedown	onmousedown	요소 위에서 마우스 버튼을 눌렀을 때 발생한다.
mouseup	onmouseup	요소 위에서 마우스 버튼을 놓을 때 발생한다.
mousemove	onmousemove	마우스 움직임이 있을 때 발생한다.

이벤트의 종류

- 키보드 입력 요소 이벤트

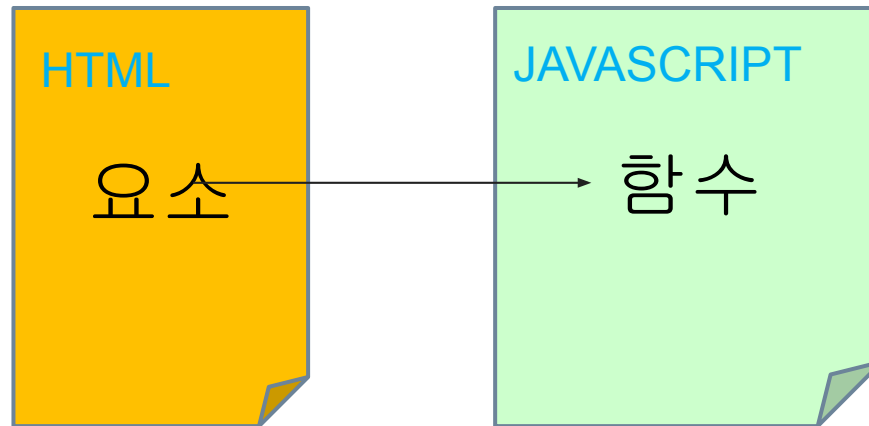
이벤트	이벤트 리스너	설명
keydown keyup	onkeydown onkeyup	사용자가 키를 눌렀다가 놓을 때 발생한다.

이벤트	이벤트 리스너	설명
focus	onfocus	사용자가 입력 요소에 키보드 입력을 시도할 때 발생한다.
submit	onsubmit	사용자가 입력 양식을 제출할 때 발생한다.
blur	onblur	포커스가 입력 요소를 벗어나는 경우에 발생한다.
change	onchange	사용자가 입력 요소의 값을 변경할 때 발생한다.

이벤트	이벤트 리스너	설명
load	onload	브라우저가 페이지 로드를 완료하면 발생한다.
unload	onunload	사용자가 현재 웹 페이지를 떠날 때 발생한다.
resize	onresize	사용자가 브라우저 윈도우의 크기를 조절할 때 발생한다.

이벤트 리스너 만들기

- 인라인 방식: HTML 요소 안에 코드 추가
- 객체의 이벤트 리스너 속성에 함수를 대입
- 객체의 `addEventListener()` 함수를 사용
익명 함수로 이벤트 리스너 작성



이벤트 리스너 만들기

- 인라인 방식

`<element event= " 자바스크립트 코드 " >`와 같이 HTML 요소에 코드를 작성해서 붙이는 방법이다.

```
<div onmouseover="this.style.backgroundColor='blue'"> Hello World!  
</div>
```

- 객체의 이벤트 리스너 속성에 함수를 대입

```
<div id="special"> Hello World! </div>  
<script>  
function changeColor() {  
    document.getElementById("special").style.backgroundColor='blue';  
}  
document.getElementById("special").onmouseover = changeColor;  
</script>
```


이벤트 리스너 만들기

- 객체의 `addEventListener()` 함수를 사용
 - DOM 객체가 가지고 있는 `addEventListener()` 함수를 이용하여 이벤트 리스너를 등록하는 방법이다. 객체의 이벤트 리스너 속성에 함수를 대입

```
let obj = document.getElementById("special");  
obj.addEventListener("mouseover", changeColor);
```

```
obj.addEventListener("mouseover", firstFunction);  
obj.addEventListener("mouseover", secondFunction);
```

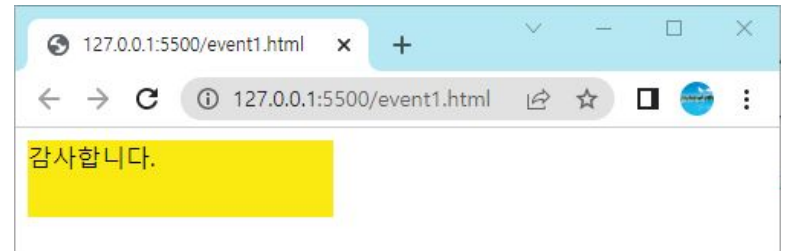
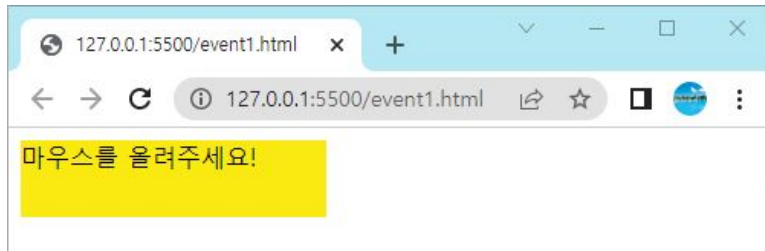
이벤트 리스너 만들기

- 익명 함수로 이벤트 리스너 작성
 - 익명 함수(이름없는 함수)나 화살표 함수를 사용하여 이벤트 리스너를 작성할 수도 있다.

```
let obj = document.getElementById("special");  
obj.addEventListener("mouseover", function() {  
  document.getElementById("special").style.backgroundColor='blue';  
});
```

예제

- 요소에 마우스가 올라오면 요소의 내용을 변경하는 코드를 작성해보자.



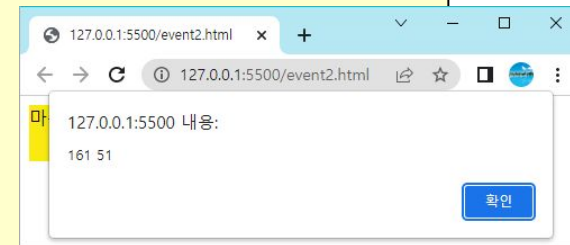
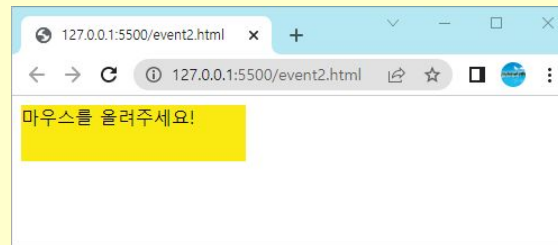
예제

```
<!DOCTYPE html>
<html>
<body>
  <div onmouseover="mouseOver(this)" onmouseout="mouseOut(this)"
    style="background-color:#faea11; width:200px; height:50px;">
    마우스를 올려주세요!</div>
  <script>
    function mouseOver(obj) {
      obj.innerHTML = "감사합니다."
    }
    function mouseOut(obj) {
      obj.innerHTML = "마우스를 올려주세요!"
    }
  </script>
</body>
</html>
```

이벤트 객체

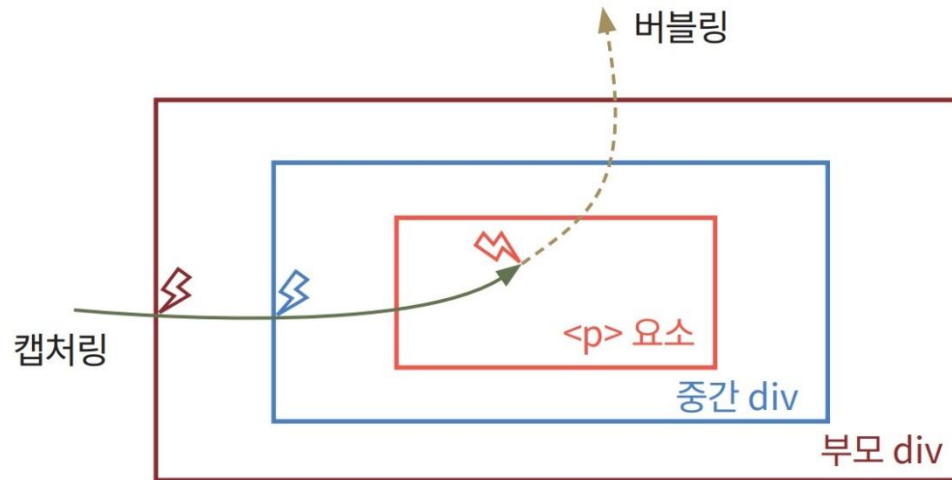
- 이벤트 리스너는 이벤트 객체(event object)를 전달받는다. 이벤트 객체는 이벤트마다 달라진다. 예를 들어서 **mouseover** 이벤트의 경우, 마우스 좌표와 마우스 버튼 정보 등이 포함된다.

```
<!DOCTYPE html>
<html>
<body>
  <div id="special" style="background-color:#faea11;width:200px;height:50px;">
    마우스를 올려주세요!</div>
  <script>
    let obj = document.getElementById("special");
    obj.onmouseover= function (e) {
      x=e.clientX;
      y=e.clientY;
      alert(x+" "+y);
      alert(e.type);
      alert(e.target);
    };
  </script>
</body>
</html>
```



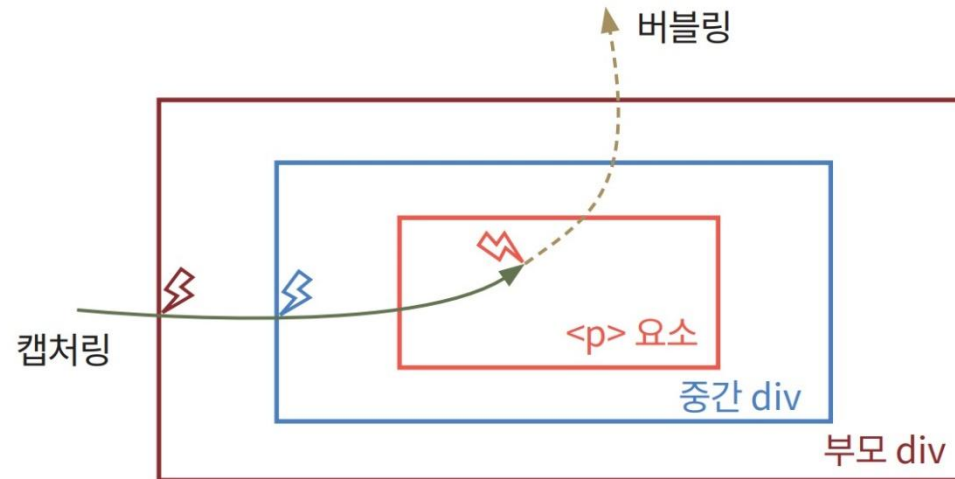
이벤트 전파

- 예를 들어서 `<div>` 요소 안에 `<p>` 요소가 있고, 사용자가 `<p>` 요소를 클릭하면 어떤 요소의 "onclick" 이벤트가 먼저 처리되어야 할까?



버블링(bubbling)과 캡처링(capturing)

- 버블링은 가장 안쪽 요소의 이벤트가 먼저 처리된 다음, 바깥쪽 이벤트가 처리된다.
- 캡처링은 가장 바깥쪽 요소의 이벤트가 먼저 처리된 다음, 안쪽 요소의 이벤트가 처리된다.



```
addEventListener(event, function, useCapture);
```

true이면 캡처링이다(디폴트는 버블링) ←

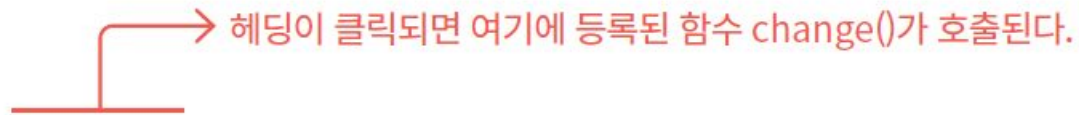
예제




```
<!DOCTYPE html>
<html lang="ko">
<head>
  <style>
    p, div {
      margin: 10px;
      border: 1px solid rgb(199, 89, 25);
    }
  </style>
</head>
<body>
  <div id="div1">DIV
    <p id="p1">P</p>
  </div>
  <script>
    let elem1= document.getElementById("div1");
    elem1.addEventListener("click", function() {alert(`캡처링 div`);}, true);
    elem1.addEventListener("click", function() {alert(`버블링 div`);});
    elem2= document.getElementById("p1");
    elem2.addEventListener("click", function() {alert(`캡처링 p`);}, true);
    elem2.addEventListener("click", function() {alert(`버블링 p`);});
  </script>
</body>
</html>
```

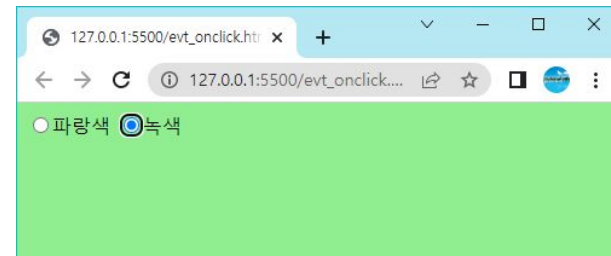
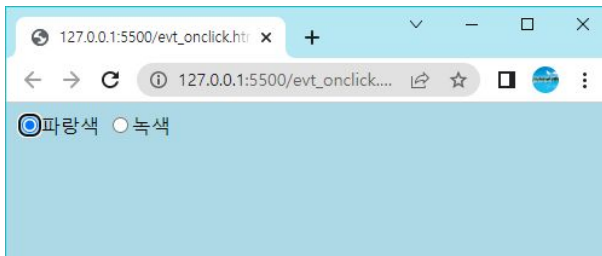
click 이벤트

- 사용자가 **HTML** 요소를 클릭하면 미리 정해놓은 코드가 실행되도록 할 수 있다.



`<h1 onclick="change()">이것은 클릭 가능한 헤딩입니다.</h1>`

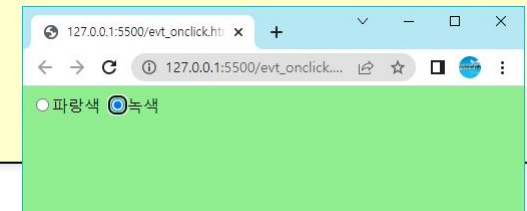
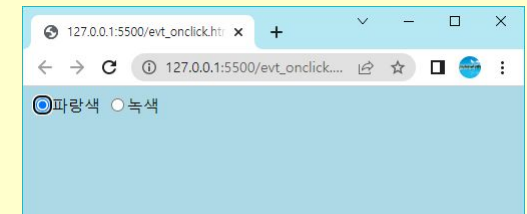
- 예제로 라디오 버튼 요소에 **click** 이벤트를 설정하여 보자. 사용자가 라디오 버튼을 클릭하면 **<body>** 요소의 색상이 변경되도록 한다.



예제

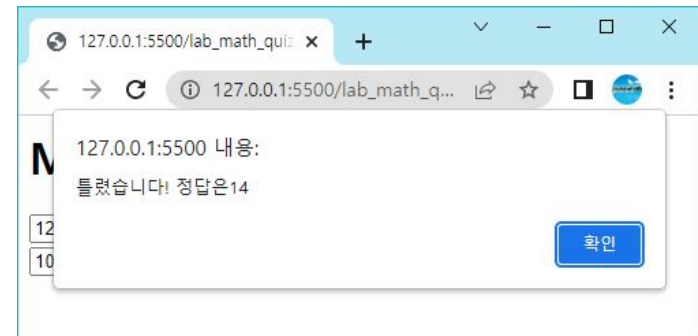
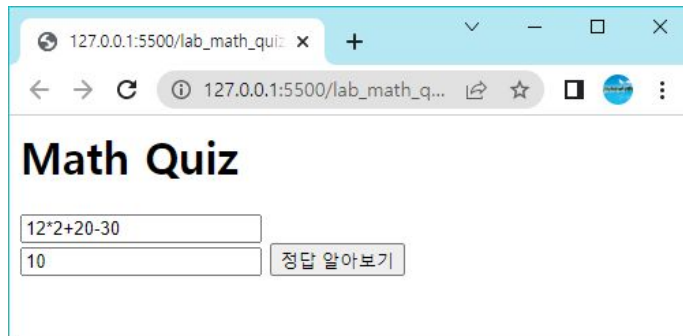
```
<!DOCTYPE html>
<html>
<head>
  <script>
    function changeColor(c) {
      document.getElementById("target").style.backgroundColor = c;
    }
  </script>
</head>
<body id="target">

  <form method="POST">
    <input type="radio" name="C1" value="v1"
      onclick="changeColor('lightblue')">파랑색
    <input type="radio" name="C1" value="v2"
      onclick="changeColor('lightgreen')">녹색
  </form>
</body>
</html>
```



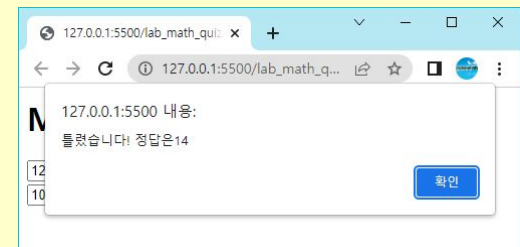
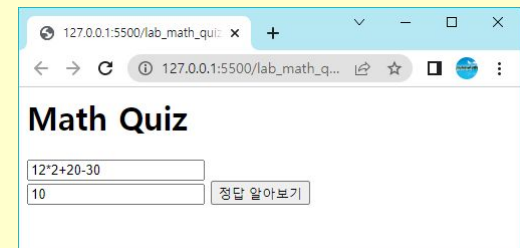
Lab: 수학 퀴즈 페이지 작성하기

- `eval()`을 사용하여 초등학생을 위한 수학 퀴즈를 출제하고 버튼을 누르면 정답인지를 체크해주는 페이지를 작성해보자.



예제

```
<html>
<head>
  <script>
    function cal() {
      let answer = document.getElementById('answer').value;
      let s = document.getElementById("problem").value;
      let solution=eval(s);
      if (answer == solution) {
        alert("정답!");
      } else {
        alert("틀렸습니다! 정답은"+solution);
      }
    }
  </script>
</head>
<body>
  <h1>Math Quiz</h1>
  <input type="text" id="problem" value="12*2+20-30"><br>
  <input type="text" id="answer">
  <button onclick="cal()">정답 알아보기 </button>
</body>
```

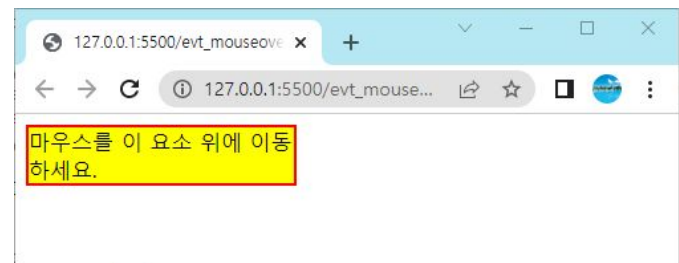
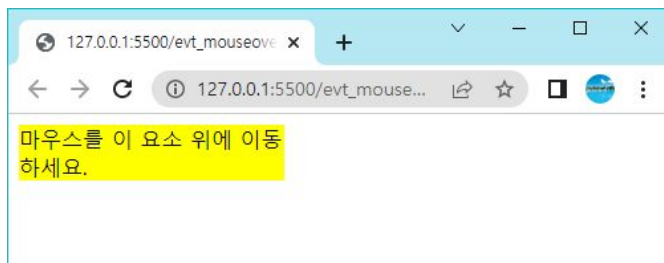


mouseover 이벤트

- mouseover와 mouseout 이벤트는 사용자가 HTML 요소 위에 마우스를 올리거나 요소를 떠날 때 발생된다.



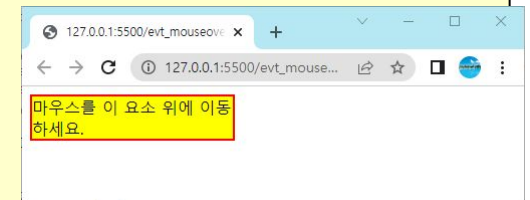
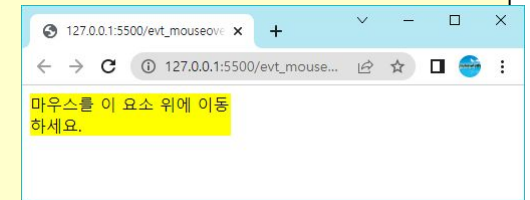
- 마우스 커서를 올리면 경계선이 표시되고 마우스 커서가 나가면 경계선이 사라지는 예제는 다음과 같이 작성한다.



예제

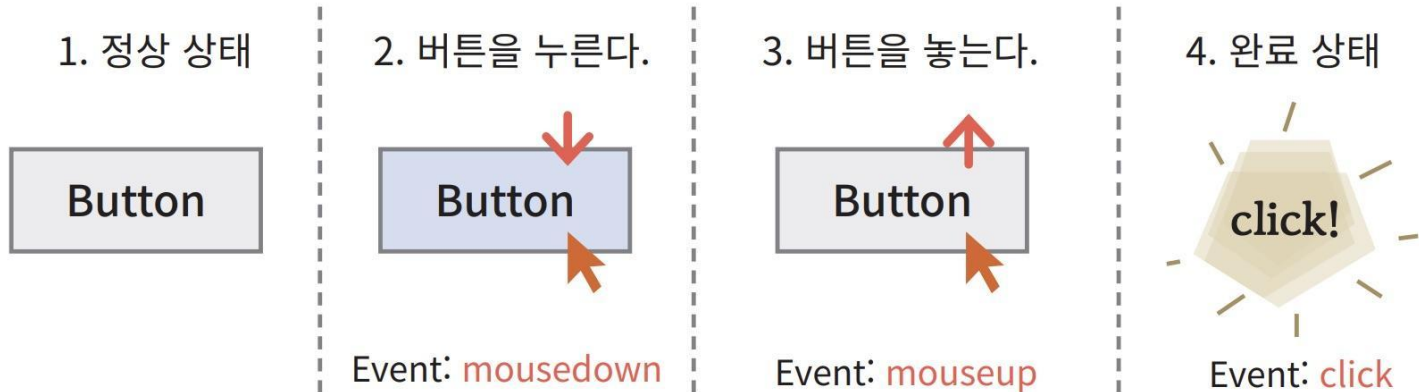
```
<html>
<head>
  <script>
    function OnMouseIn(elem) {
      elem.style.border = "2px solid red";
    }
    function OnMouseOut(elem) {
      elem.style.border = "";
    }
  </script>
</head>

<body>
  <div style="background-color: yellow; width: 200px"
    onmouseover="OnMouseIn (this)" onmouseout="OnMouseOut (this)">
    마우스를 이 요소 위에 이동하세요.
  </div>
</body>
</html>
```



mousedown, mouseup 이벤트

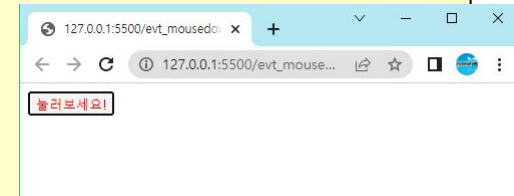
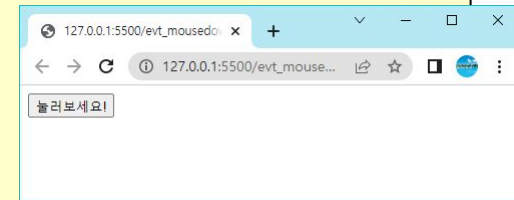
- 먼저 마우스 버튼이 클릭되면 **mousedown** 이벤트가 발생한다. 이어서 마우스 버튼이 떼어지면 **mouseup** 이벤트가 발생하고 마지막으로 마우스 클릭이 완료되면서 **click** 이벤트가 발생한다.



예제

- 마우스 커서를 올리면 경계선이 표시되고 마우스 커서가 나가면 경계선이 사라지는 예제는 다음과 같이 작성한다.

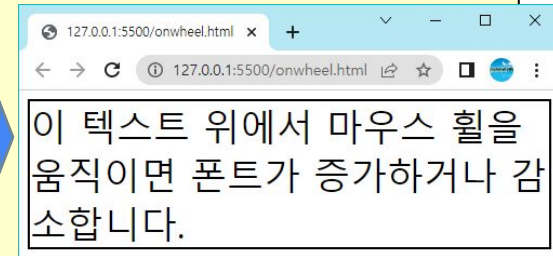
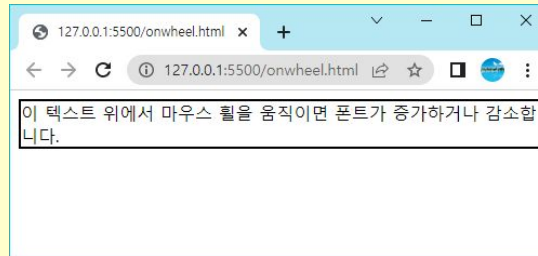
```
<html>
<head>
  <script>
    function OnButtonDown(button) {
      button.style.color = "#ff0000";
    }
    function OnButtonUp(button) {
      button.style.color = "#000000";
    }
  </script>
</head>
<body>
  <button onmousedown="OnButtonDown (this)" onmouseup="OnButtonUp
(this)">눌러보세요!</button>
</body>
```



wheel 이벤트

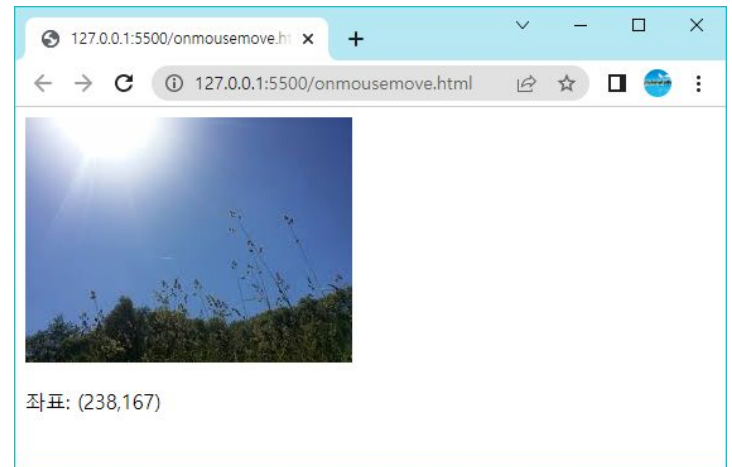
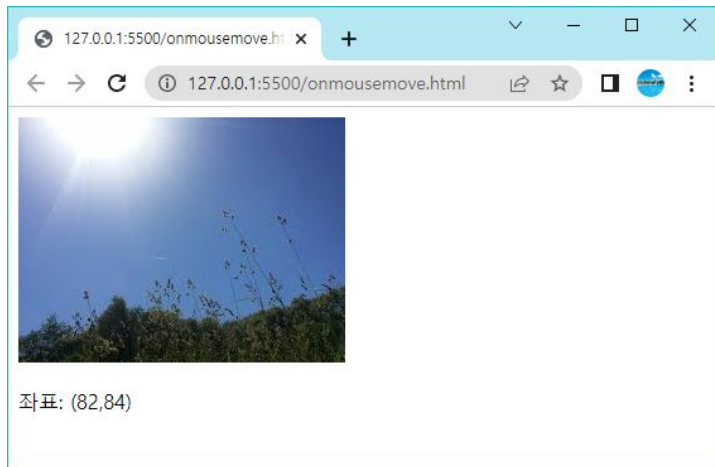
- 마우스의 휠을 돌리면 **wheel** 이벤트가 발생한다. **<div>** 요소 위에서 마우스 휠을 조작하면 폰트가 증가되는 예제를 작성해보자.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    #div1 {
      border: 2px solid black;
    }
  </style>
</head>
<body>
  <div id="div1">이 텍스트 위에서 마우스 휠을 움직이면 폰트가 증가하거나 감소합니다.</div>
  <script>
    document.getElementById("div1").addEventListener("wheel", sub);
    let size = 30;
    function sub(e) {
      if (e.wheelDelta > 0)
        size++;
      else
        size--;
      this.style.fontSize = size + "px";
    }
  </script>
</body>
</html>
```



onmousemove 이벤트

- `onmousemove` 이벤트는 마우스의 좌표를 우리에게 전달해준다. 마우스가 움직이면 연속하여 발생된다.



예제

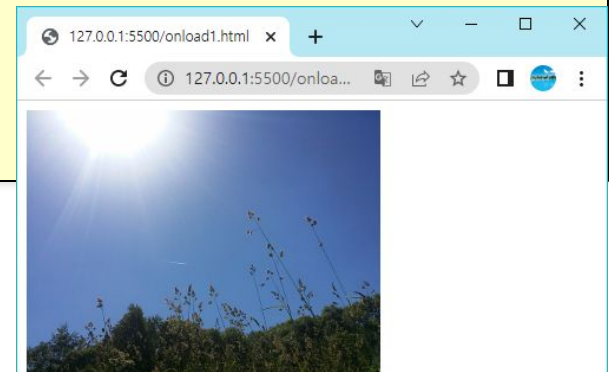
```
<!DOCTYPE html>
<html>
<head>
  <style>
    #img1 {
      width: 200px;
      border: 1px solid black;
    }
  </style>
</head>
<body>
  <br>
  <p id="test"></p>
  <script>
    function sub(e) {
      let x = e.clientX;
      let y = e.clientY;
      let coor = "좌표: (" + x + "," + y + ")";
      document.getElementById("test").innerHTML = coor;
    }
  </script>
</body>
</html>
```

load 이벤트

- HTML 문서가 로딩되거나, 사용자가 웹페이지를 떠나면 각각 **load**와 **unload** 이벤트가 발생된다. **load** 이벤트는 문서의 로딩이 완료되면 발생하는 이벤트이다.
- 자바스크립트 코드에서 아직 생성되지 않은 이미지나 요소를 미리 사용하면 안 되기 때문이다.

```
<!DOCTYPE html>
<html lang="en">
<body>
  
  <script>
    document.getElementById('result').innerHTML
      = "이미지의 폭="+ document.getElementById('image').clientWidth;
  </script>
  <div id="result"></div>
</body>
</html>
```

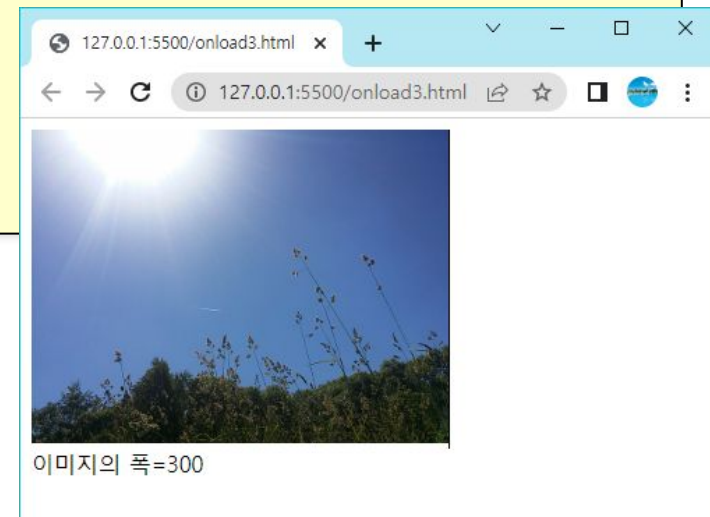
나타나지 않는다!



load 이벤트

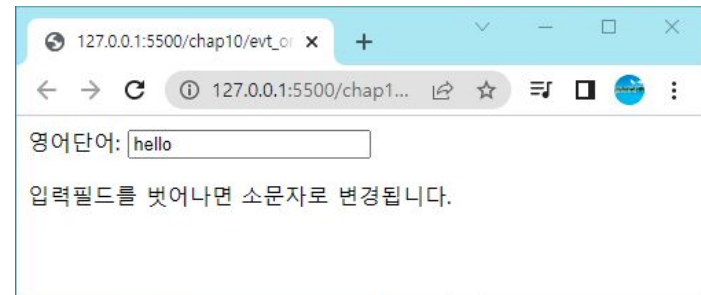
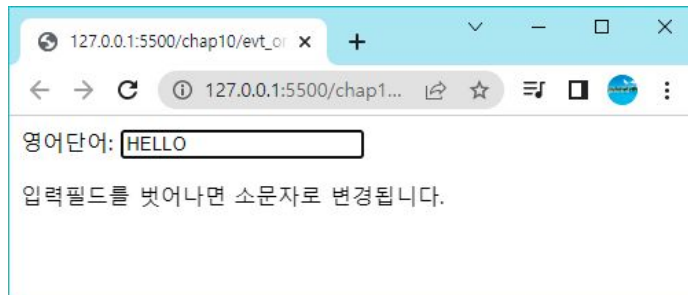
```
<!DOCTYPE html>
<html lang="en">
<body>

  
  <script>
    function loadImage() {
      document.getElementById('result').innerHTML
        = "이미지의 폭=" +
        document.getElementById('image').clientWidth;
    }
  </script>
  <div id="result"></div>
</body>
</html>
```



change 이벤트

- **change** 이벤트는 입력 필드를 검증할 때 종종 사용된다. 아래 예제에서 사용자가 입력 필드의 내용을 변경하면 `toLowerCase()` 함수가 호출된다.

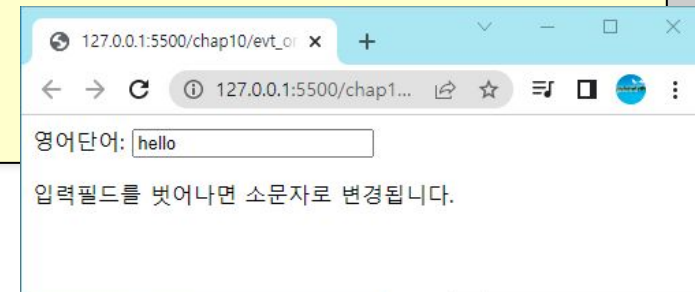


change 이벤트

```
<!DOCTYPE html>

<html>
<head>
<script>

    function sub()    {
        let x = document.getElementById("name");
        x.value = x.value.toLowerCase();
    }
</script>
</head>
<body>
영어단어: <input type="text" id="name" onchange="sub()">
<p>입력필드를 벗어나면 소문자로 변경됩니다.</p>
</body>
</html>
```



입력값의 유효성 검증

- 자바스크립트는 사용한 입력한 데이터를 서버로 보내기 전에, 클라이언트 컴퓨터에서 사전 검증하는데 많이 사용된다.
- 다음과 같은 회원 가입 페이지에서 아이디를 입력할 때, 정해진 문자 수를 초과할 수도 있고, 이메일 주소를 잘못 입력할 수도 있다.

```
<h3>회원가입</h3>
```

```
<form>
```

```
이름: <input type='text' id='name' /><br />
```

```
주소: <input type='text' id='addr' /><br />
```

```
생일: <input type='date' id='birthday' /><br />
```

```
아이디(6-8 문자):
```

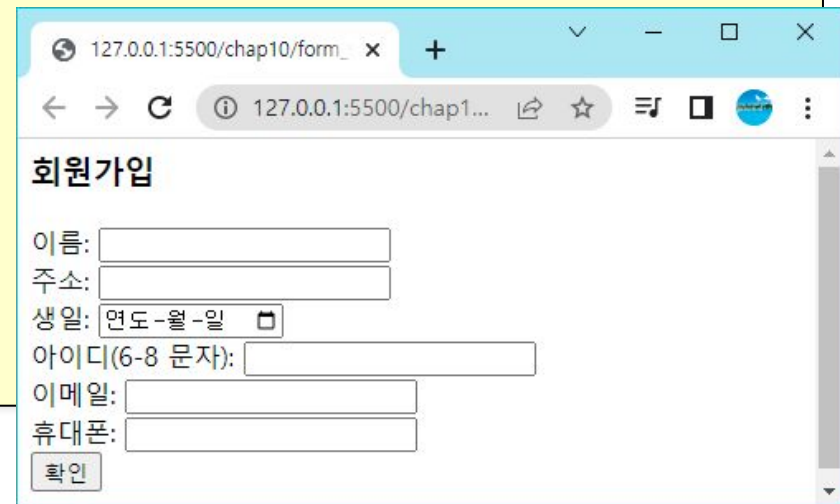
```
<input type='text' id='username' /><br />
```

```
이메일: <input type='email' id='email' /><br />
```

```
휴대폰: <input type='tel' id='phone' /><br />
```

```
<input type='submit' value='확인' /><br />
```

```
</form>
```



127.0.0.1:5500/chap10/form_ x +

127.0.0.1:5500/chap1...

회원가입

이름:

주소:

생일: 

아이디(6-8 문자):

이메일:

휴대폰:

검증내용

- 사용자가 필수적인 필드를 채웠는가?
- 사용자가 유효한 길이의 텍스트를 입력하였는가?
- 사용자가 유효한 이메일 주소를 입력하였는가?
- 사용자가 유효한 날짜를 입력하였는가?
- 사용자가 숫자 필드에 텍스트를 입력하지 않았는가?

입력 양식 데이터 접근

- 사용자가 입력한 입력 양식 데이터에 접근하기 위해서는 입력 양식 안에 있는 필드의 **id**나 **name** 속성을 이용하여야 한다.

id 속성은 웹 페이지에서 요소들을 식별한다. →

name 속성은 입력 양식 내부에서 필드들을 식별한다. →

```
<input type="text" id="address" name="addr" />
```

주소:

```
<input type="text" name="addr" onclick="display(this.form)"/>
```

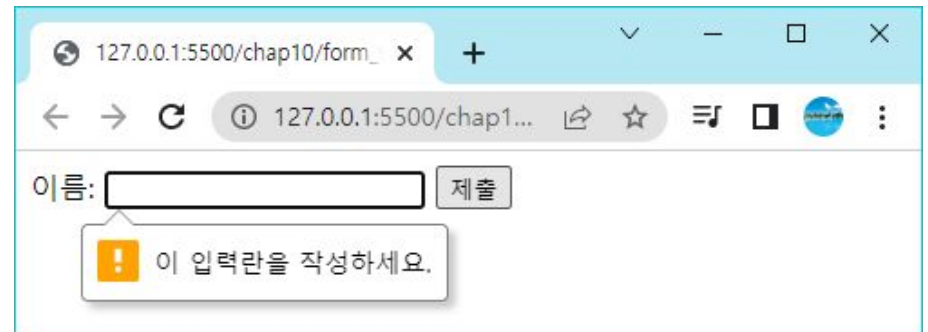
```
function display(form) {  
    alert(form["addr"].value);  
}
```

form 객체는 입력양식 안의 필드로 이루어진 배열로서 name을 식별자로 하여 필드값을 저장한다.

공백 검증

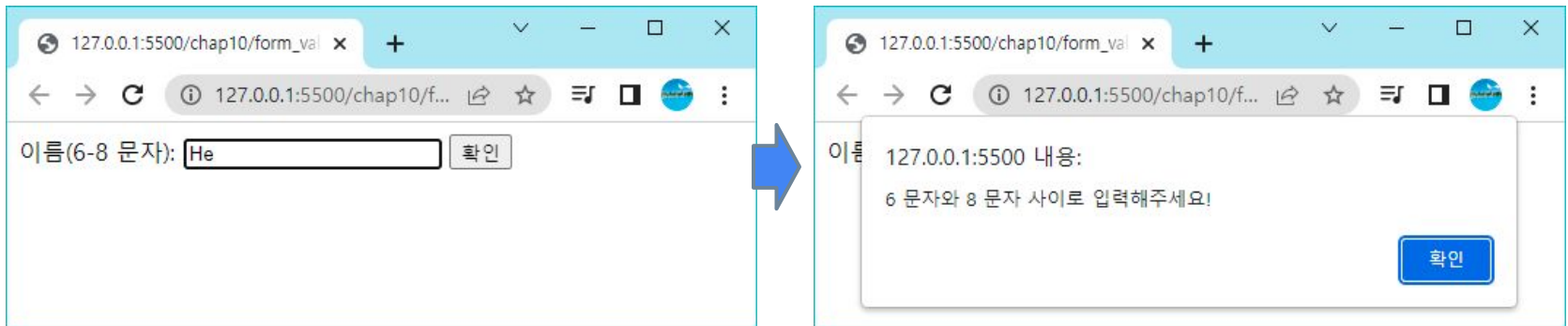
- 가장 기초적인 데이터 검증은 필드가 비어있는지를 체크하는 것이다.

```
...  
<form>  
  이름: <input type='text' id='user' required >  
  <input type='submit' value='제출'>  
</form>
```



데이터 길이 검증

- 사용자가 정해진 개수의 문자만을 입력하도록 하는 경우도 많다. 예를 들어서 주민등록번호는 13자리이다.

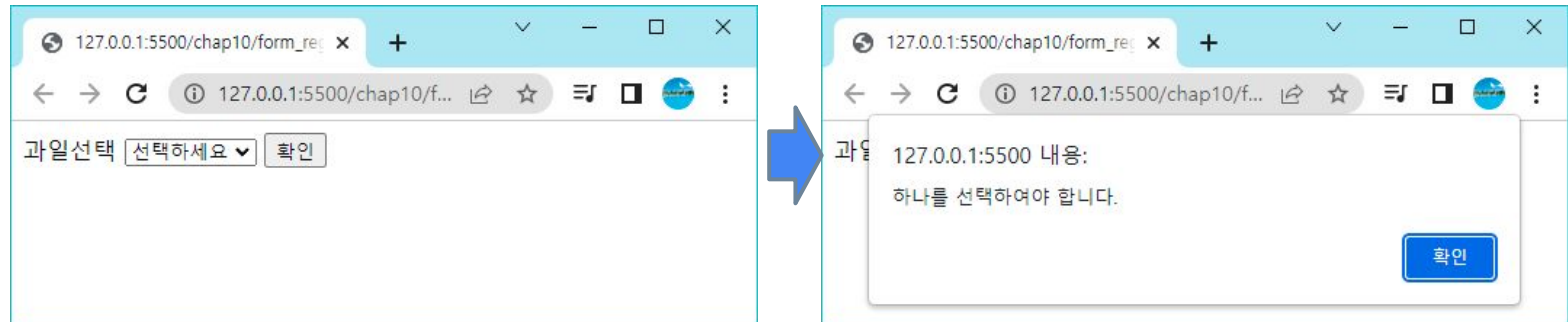


데이터 길이 검증

```
<script>
  function checkLength(elem, min, max) {
    let s = elem.value;
    if (s.length >= min && s.length <= max) {
      return true;
    } else {
      alert(min + " 문자와 " + max + " 문자 사이로 입력해주세요!");
      elem.focus();
      return false;
    }
  }
</script>
<form>
  이름(6-8 문자): <input type='text' id='name' />
  <input type='button'
    onclick="checkLength(document.getElementById('name'), 6, 8)"
    value='확인' />
</form>
```

선택 검증

- HTML **select** 요소에서 사용자가 선택을 했는지를 검증하려면 약간의 트릭이 필요하다. 즉 첫 번째 옵션을 도움말로 두는 것이다.

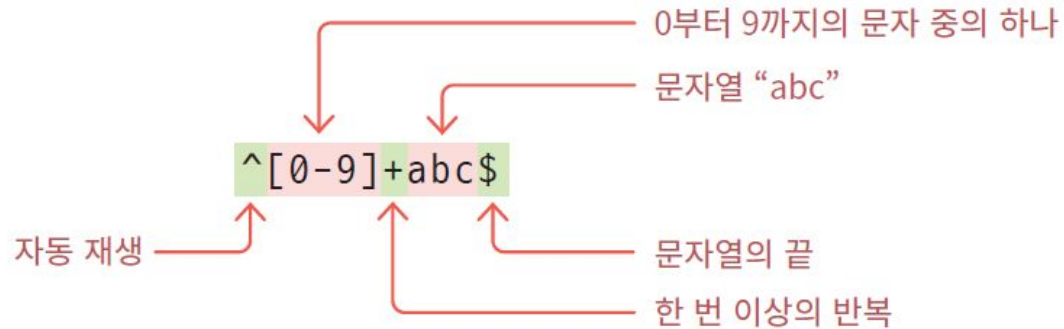


데이터 길이 검증

```
<script>
  function checkSelection(elem, msg) {
    if (elem.value == 0) {
      alert(msg);
      elem.focus();
      return false;
    } else {
      return true;
    }
  }
</script>
<form>
과일선택 <select id="fruits" class="required">
  <option value="0">선택하세요</option>
  <option value="1">사과</option>
  <option value="2">배</option>
  <option value="3">바나나</option>
</select>
<input type="button"
  onclick="checkSelection(document.getElementById('fruits'), '하나를 선택하여야 합니다.')"
  value='확인' />
</form>
```


정규식

- 정규식(regular expression)이란 특정한 규칙을 가지고 있는 문자열들을 표현하는 수식이다.



정규식

식	기능	설명
^	시작	문자열의 시작을 표시
\$	끝	문자열의 끝을 표시
.	문자	한 개의 문자와 일치
\d	숫자	한 개의 숫자와 일치
\w	문자와 숫자	한 개의 문자나 숫자와 일치
\s	공백문자	공백, 탭, 줄바꿈, 캐리지리턴 문자와 일치
[]	문자 종류, 문자 범위	[abc]는 a 또는 b 또는 c를 나타낸다. [a-z]는 a부터 z까지 중의 하나, [1-9]는 1부터 9까지 중의 하나를 나타낸다.

수량 한정자	기능	설명
*	0회 이상 반복	"a*"는 "", "a", "aa", "aaa"를 나타낸다.
+	1회 이상	"a+"는 "a", "aa", "aaa"를 나타낸다.
?	0 또는 1회	"a?"는 "", "a"를 나타낸다.
{m}	m회	"a{3}"는 "aaa"만 나타낸다.
{m, n}	m회 이상 n회 이하	"a{1, 3}"는 "a", "aa", "aaa"를 나타낸다.
(ab)	그룹화	(ab)*은 "", "ab", "abab" 등을 나타낸다.

정규식의 예

- 만약 계좌번호를 검증한다고 가정하자(계좌번호는 10자리의 숫자로 되어 있다고 가정하자). 이것은 정규식 `/^\d{10}$/`으로 나타낼 수 있다.

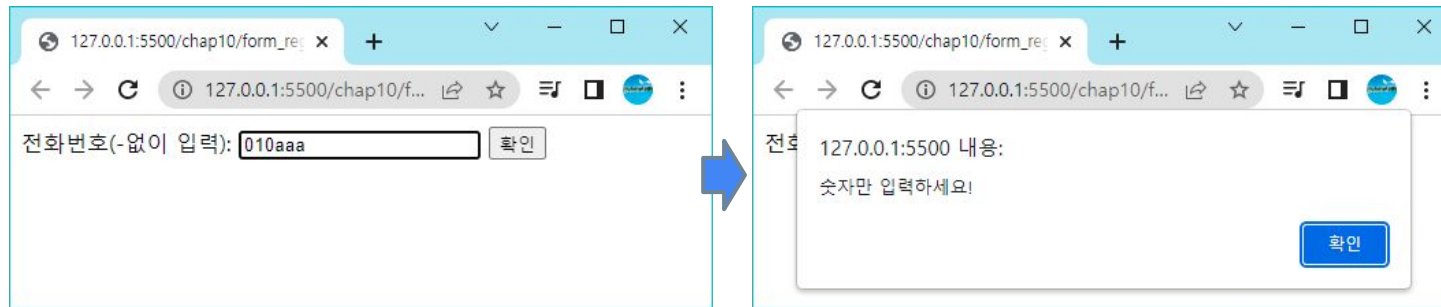
```
let exp = /^\d{10}$/;
if( !exp.test(field.value) ){
    alert("오류!!");
    return false;
}
```

→ 이 리터럴은 RegExp 객체를 생성한다.

→ RegExp 객체의 test() 메소드는 일치하면 true를 반환한다.

예제: 숫자 여부 검사하기

- 숫자로만 된 데이터를 검증하는 가장 좋은 방법은 정규 표현식을 사용하는 것이다. `/^[0-9]+$ /`은 문자열이 모두 숫자로만 되어 있어야 일치한다



예제 소스

```
<script>
  function checkNumeric(elem, msg) {
    let exp = /^[0-9]+$/;
    if (elem.value.match(exp)) {
      return true;
    } else {
      alert(msg);
      elem.focus();
      return false;
    }
  }
</script>
<form>
전화번호(-없이 입력): <input type='text' id='phone' />
<input type='button'
  onclick="checkNumeric(document.getElementById('phone'), '숫자만 입력하세요!')"
  value='확인' />
</form>
```

Q & A

